ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

Corso di Laurea in Ingegneria e Scienze Informatiche

Progettazione di un configuratore di vendite

Tesi di laurea in:
Basi di dati

Relatore
Prof.sa Annalisa Franco

Candidato
Cavina Mattia

III Sessione di Laurea
Anno Accademico 20252026

Sommario

To be written.

Indice

Elenco delle figure

List of Listings

6.1 Esempio di query LINQ 29

Capitolo 1

Introduzione

Il progetto di tesi è volto alla progettazione ex-novo del programma di vendita dell'azienda Cepi S.p.a., azienda con una produzione di tipo **ETO!** (**ETO!**) specializzato nella progettazione di impianti di stoccaggio custom. La tipologia di produzione **ETO!** ha portato ad un profondo studio della situazione odierna, delle sue esigenge attuali e future tenendo aperte le possibilità di ampliamenti. L'analisi ha comportato un sostanziale lavoro di programmazione e racccolta dati per coprire nel modo più ampio i bisogni aziendali. Questa prima fase è stata condotta inizialmente attraverso la selezione di figure chiave scelte per mansione o/e ruolo, che potessero dare una visione dei bisogni del proprio o altrui reparto e successivamente sottoponendoli ad interviste. Sono state condotte 14 interviste, di cui 8 in modalità vis a vis, mentre le restanti 6 attraverso la dimostrazione dell'attuale applicativo utilizzato. Il risultato delle interviste condotte, ha fatto emergere attuali e nuove necessità architteturali da affrontare come il cross-platform, l'utilizzo offline del database e la necessità di costrurire offerte in modo più agile e strutturato. Questa prima fase ha inoltre sottolineato come il progetto non debba solo sostituire l'attuale sistema, ma debba inserirsi in un contesto di rinnovamento più ampio, coordinadosi con altri programmi aziendali e relativi database in ottica "**SRP!** (**SRP!**)" eleggendo dei master data che siano i soli resposabili della creazione di certi dati. Il **SRP!** ha plasmato notevolmente la costruzione del database ricreando, dove possibile, le strutture pre-esistenti da cui si necessitava recuperare i dati mediante reverse engineering. **??** L'intero sistema è stato pensato per ap-

poggiarsi all'infrastruttura esistente aziendale, come SQLServer[**?**], ma studiando delle logiche di flessibilità e ampliamento dinamico studiando accuratamente le relazioni tra le tabelle. Si è reso necessario cercare e scegliere inoltre un framework per la parte grafica che potesse soddisfare il deployment cross-platform. Questo requisito è emerso in modo chiaro dalle interviste, in quanto ci sono situazioni in cui il personale aziendale si trova a dover lavorare in mobilità, come le fiere di settore. La costruzione delle interfacce grafiche, è stata supportata dal sito "Figma"[**?**] che ha permesso una progettazione in ottica mobile first.**??** La scelta accurata dei framework e le strutture dati nella fase iniziale, ha pesato molto in termini di tempo e progettazione, per garantire al richiedente un sistema che non rischiasse né il debito tecnico, né la preclusione a sviluppi futuri. La pianificazione dei primi test con gli utenti, dei rilasci delle demo e del metodo di progettazione sono state cruciali per dettare tempi e priorità degli item dovendo coprire in modo esaustivo almeno il core del progetto.

Capitolo 2

Situazione As-Is

2.1 Premessa

La situazione infrastrutturale aziendale è stata studiata dal sottoscritto nel corso del tirocinio formativo. La raccolta dei dati è stata svolta mediante interviste e studio dell'attuale sistema informatico. Nelle sezioni che seguono introdurrò brevemente l'As-Is.

2.2 Situazione infrastrutturale aziendale

L'infrastruttura informatica aziendale si appoggia su server fisici allocati internamente e gestiti mediante Active Directory. I server fisici sono, a loro volta virtualizzati per gestire separatamente le memorie in base alla finalità garantendo isolamento delle informazioni e permessi di accesso. Ogni reparto ha accesso a determinati server virtuali in cui gestisce in autonomia i propri contenuti. I database dei principali programmi sono gestiti all'interno di un server dedicato dove è installato un motore SQLServer.[**?**]. I principali database aziendali sono:

- **AdHoc[?]:** l'**ERP!** (**ERP!**) aziendale, programma che gestisce le attività aziendali come gli approvvigionamenti e la movimentazione dei prodotti, le vendite, le fatturazioni nonché i cicli produttivi.

- **Fusion[?]:** il **PLM!** (**PLM!**) aziendale, programma volto alla gestione dei prodotti meccanici, dalla loro creazione all'archiviazione dei disegni con relative distinte dei componenti. Altre funzioni applicate sui prodotti sono la gestione delle revisioni o versionamenti, ed infine la loro fine produzione.
- **BombConfine:** database ponte tra i database esterni e quello dell'**ERP!**. All'interno sono presenti tabelle di frontiera scritte da programmi esterni e che il gestionale usa per importare i dati.

La sicurezza aziendale è sostenuta da **NAT!** (**NAT!**) e Firewall che impediscono l'accesso dall'esterno se non tramite **VPN!** (**VPN!**). L'infrastruttura aziendale è visibile in figura **??**.

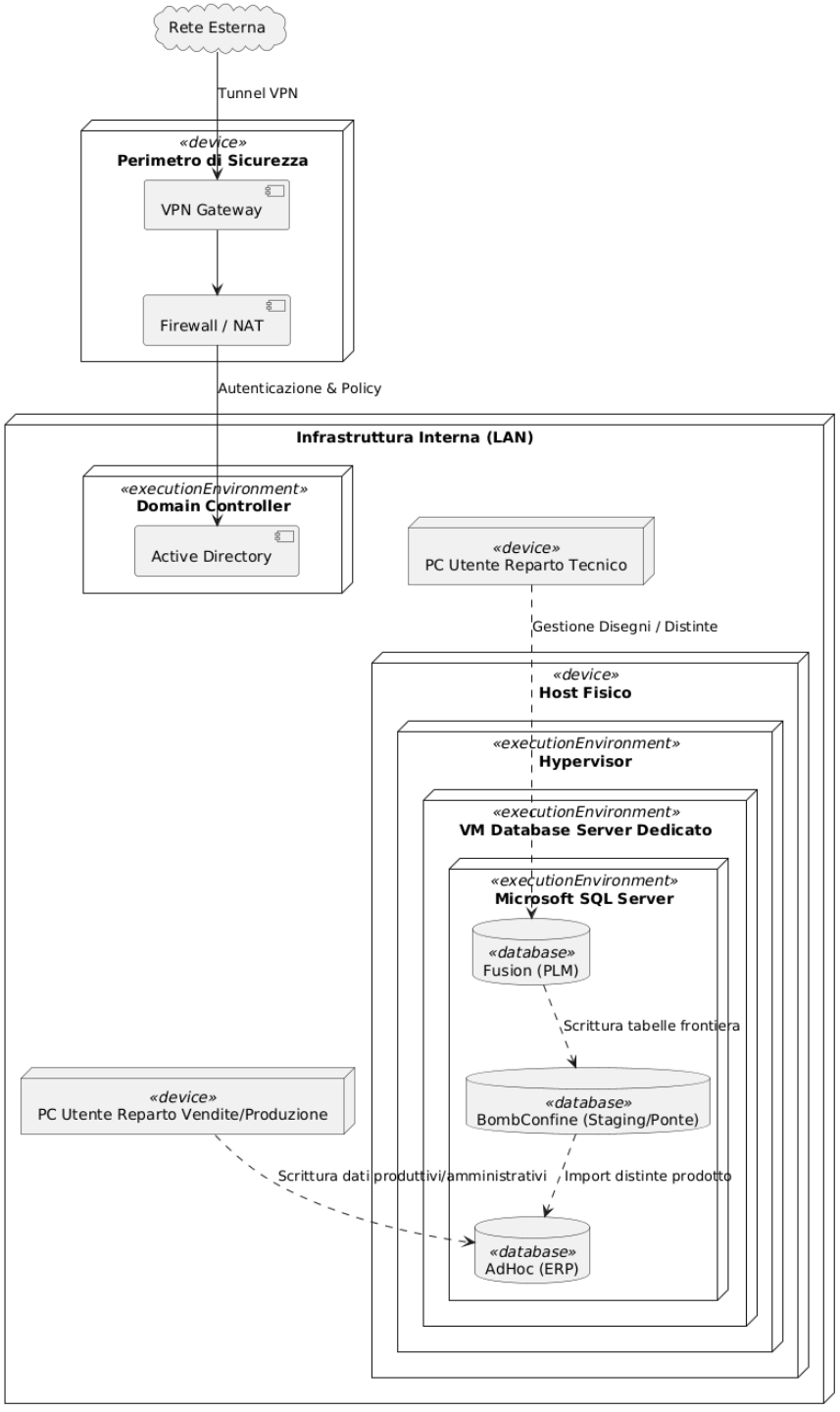


Figura 2.1: Schema infrastruttura aziendale.

2.3 Attuali software utilizzati

Nel capitolo precedente abbiamo fatto cenno ai database ed ai relativi programmi che gestiscono quanto concerne l'**ERP!** e il **PLM!**. Per quanto riguarda le vendite, punto focale dell'intervento richiesto, l'azienda utilizza attualmente un'applicazione sviluppata internamente. L'attuale sistema sviluppato in Access, programma della suite Microsoft Office, comprende al suo interno sia il frontend che il backend. Il software è stato implementato circa 30 anni fa ed ha subito numerosi aggiornamenti e modifiche nel tempo per adattarsi alle esigenze commerciali e all'evoluzione del mercato. L'applicazione consente di creare, modificare e gestire i contratti con i clienti, includendo funzionalità quali la gestione degli sconti, la personalizzazione delle offerte e la stampa dei documenti. Tuttavia, nonostante le funzionalità implementate, il software presenta significative criticità che ne limitano l'efficienza e l'usabilità, come confermato dalle interviste condotte con gli utenti. Tali criticità includono instabilità operativa, mancanza di standardizzazione, codifiche obsolete, difficoltà negli aggiornamenti di prezzo e assenza di integrazione con altri sistemi aziendali. Di conseguenza, è indispensabile sviluppare un nuovo programma che affronti tali problematiche e migliori l'esperienza degli utenti, preservando nel contempo le funzionalità essenziali del sistema attuale. Il programma Access è stato fondamentale in questi anni per l'utilizzo in zone offline. Questa funzionalità è stata favorita dalla natura del programma stesso, il quale racchiude in un solo file database e frontend, copiandolo in locale e sincronizzandolo una volta tornato in sede.

2.4 Organizzazione aziendale e flussi di lavoro

Il flusso lavorativo è risultato non essere un aspetto secondario nello sviluppo richiesto. Esso si deve interporre tra il reparto commerciale, ovvero addetti alle vendite, i reparti progettuali e produttivi come uffici tecnici, logistica ed amministrazione. L'attuale flusso di lavoro analizzato pone il reparto commerciale come punto di partenza dell'intero flusso che arriva fino al post-vendita. Il contratto, creato da tale reparto, dopo essere stato stipulato viene inviato al reparto amministrativo che ne importa i dati all'interno del gestionale. Simultaneamente esso

viene mandato al reparto project manager che lo revisiona tecnicamente in caso di errori progettuali. Dopo la revisione tecnica, il contratto viene spedito al reparto logistico che preventivamente verifica i dati di destinazione ed organizza le spedizioni sulla base del venduto. Il flusso di lavoro può essere visto in figura **??**. A causa del sistema obsoleto utilizzato ed il debito tecnico accumulato negli anni, l'attuale sistema è isolato rispetto ai programmi utilizzati dall'azienda. Quest'ultimo punto fa parte delle richieste imprescendibili da risolvere. L'integrazione nell'ecosistema infomatico passa prima di tutto dall'unificazione dei codici, ovvero quelli a listino con i prodotti a gestionale, ma anche creando un flusso di notifiche tra i vari ambiti.

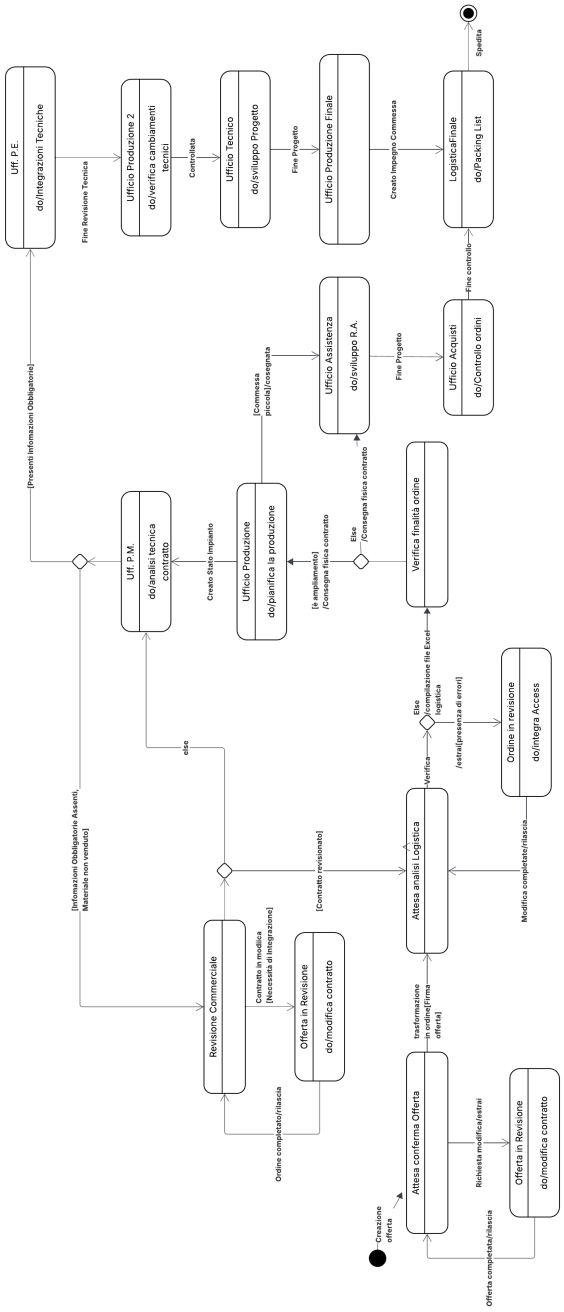


Figura 2.2: Schema degli stati di una offerta commerciale

Capitolo 3

Raccolta dei requisiti

3.1 Scelta dei candidati

La raccolta requisiti si è svolta mediante la selezione di figure aziendali con distinti ruoli ed esperienza. Queste figure selezionate sono state poi sottoposte ad una intervista sia in modalità vis a vis, che in alcuni casi da remoto per favorire la registrazione del meeting. I candidati possono essere raggruppati in soci aziendali, utilizzatori, e non utilizzatori. All'interno degli ultimi due gruppi sono state selezionate, per far fronte ai basias dovuti all'abitudine, candidati sia con esperienza pluriennale che neoassunti. In totale sono state intervistate 14 persone così suddivise:

Utilizzatori abituali:

- **Agenti commerciali esterni:** 2 persone
  - 1 agente con esperienza limitata.
  - 1 agente con esperienza consolidata.
- **Commerciali interni:** 4 persone
  - 1 specializzato in nuovi impianti.
  - 1 specializzato in ricambi.

  - 2 con competenze trasversali, sia per quanto concerne nuovi impianti che post-vendita.

Utilizzatori saltuari:

- **Soci aziendali con responsabilità:** 3 persone
  - 1 responsabile delle vendite.
  - 1 responsabile tecnico.
  - 1 responsabile amministrativo.
- **Direttore di produzione:** 1 persona

Non utilizzatori:

- **Impiegati tecnici:** 4 persone
  - 1 Project manager.
  - 1 Responsabile logistica.
  - 1 Progettista meccanico.
  - 1 Progettista elettrico.

L'ordine delle interviste svolte non è stato lineare, ma si è optato di partire dalla proprietà, per capire la direzione ed aspettative del progetto. Le restanti interviste hanno seguito un ordine logico, ma al contempo sparso, alternando utilizzatori e non utilizzatori, con l'obbiettivo di non saturare ogni singolo reparto. Questa modalità, ha consentito di fare da guida per le interviste successive.

3.2 Modalità di svolgimento interviste

Le interviste, come esplicato in precedenza, si sono svolte singolarmente sia in presenza che da remoto. Con il permesso degli interlocutori sono state registrate, anche solo l'audio, per poi essere rielaborate e trascritte con supporto di **IA!** (**IA!**) generativa. La scelta di utilizzare la modalità fisica o da remoto è stata

dettata dalla necessità. Per gli utilizzatori abituali si è reso necessario registrare visivamente come utilizzano l'attuale applicativo. Tramite questa modalità si è potuto raccogliere i flussi di utilizzo e le principali funzioni presenti da mantenere. Nei restanti incontri si è preferito un approccio in presenza con l'interlocutore, facilitato la raccolta di requisiti e consigli sul progetto.

Le prime interviste sono state supportate da una scaletta di domande, preparate sulla base dell' AS-IS **??** che hanno guidato l'incontro. Con il progredire delle interviste, è stato adottato un approccio più flessibile che ha affinato i risultati delle interviste in previsione delle successive. Questa modalità ha consentito di ricavare i requisiti fondamentali ordinati per importanza.

3.3 Risultati interviste condotte

3.3.1 Rispetto all'attuale sistema

Funzionalità da mantenere

Dalle interviste condotte, è emersa l'importanza di mantenere una serie di funzionalità, quali:

- Suddivisione del contratto in capitoli che raggruppano porzioni di esso.
- Gestione delle scontistiche a cascata, ovvero la possibilità di inserirle rispetto alla singola voce contrattuale, nel capitolo, o dell'offerta stessa.
- Gestione di voci o capitoli opzionali, che non concorrono al prezzo offerto, ma ad uno opzionale.
- I campi del database che attualmente concorrono alla creazione dei report, sono da mantenere.
- Creazione reportistica formato PDF del contratto digitale. Da gestire, come ora, il multiligua del report stesso.
- Esportazione all'**ERP!** delle voci contrattuali presenti nell'ordine.

Criticità emerse

Mediante le dimostrazioni di utilizzo si è potuto raccogliere le criticità da attenzionare, come:

- Blocchi frequenti in condizione di concorrenza. Se più utenti utilizzano in contemporanea il file si generano deadlock.
- Non è presente una gestione centralizzata dei listini prodotto. Attualmente si gestisce esternamente tramite file Excel.
- Mancanza di linee guida nella compilazione delle offerte. Si è avanzata l'ipotesi di un configuratore di prodotto che aiuti l'utente.
- Mancanza di gestione dei valori duplicati e controlli sulle foreign key porta a creare clienti e prodotti duplicati.

3.3.2 Richeste avanzate

Gestione ruoli e permessi

La scelta di coprire quanti più reparti possibili, ha fatto emergere la mancanza di gestione dei ruoli. Si è richiesto quindi di attribuire ad ogni utente un ruolo, a sua volta ognuno di essi avrà viste sul database e permessi dedicati. I ruoli individuati sono:

- Admin con visione sull'intero sistema, con possibiltà esclusiva di modifica ad alcune tabelle fondamentali.
- Commerciali interni che possono gestire totalmente i clienti e contratti.
- Commerciali esterni all'azienda, esempio agenti a partita IVA, possono vedere solo i propri clienti e contratti.
- Amministrazione contabile, può modificare i dati contabili dei clienti. Questo ruolo può altresi esportare le voci contrattuali all'**ERP!** per gestire la fiscalità.

- Project Manager con compito di revisionare tecnicamente il contratto. Questo compito è volto alla gestione a monte dell'invenduto.
- Visualizzatore senza permessi di scrittura.

Gestione dei contratti

Nella creazione dei contratti, core del progetto, si dovrà sviluppare il vero salto di qualità. Nelle interviste si è cercato di raccogliere le richieste che, anche dai primi rilasci, potessero dare al richiedente un sistema più ottimale. Tali richieste sono:

- Gestione storico dei dati in caso di modifica.
- Gestione di più versioni di uno stesso contratto legati dal medesimo codice progressivo. In caso di approvazione serve far decadere le versioni non approvate.
- Importazione dal **PLM!** di dati tecnici dei prodotti. Questi dati sono fondamentali nei calcoli utili alla contrattazione (Es: Potenza elettrica totale consumata).
- Gestione degli stati di avanzamento di un progetto e relative notifiche.
- Revisione tecnica del venduto in modo da dare la possibilità di proporre all'occorrenza integrazioni contrattuali.

Capitolo 4

Analisi dei requisiti

4.1 Requisiti funzionali

Questa sezione è dedicata alla suddivisione dei dati raccolti in requisiti funzionali. La tabella, nella figura **??** seguente, suddivide i requisiti per area del progetto. Insieme al requisito e alla sua descrizione è presente anche l'elenco dei ruoli a cui esso è rivolto.

Area / Modulo	Requisito Funzionale	Descrizione / Specifiche	Ruoli Coinvolti
Sicurezza & Accessi	Profilazione RBAC granulare	Gestione permessi per tabella (lettura/scrittura) tramite tabelle utenti e utenti_tipi.	Tutti i ruoli
Anagrafica Clienti	Controllo anti-duplicazione	Validazione obbligatoria dell'univocità tramite Partita IVA in fase di inserimento.	Amministrazione, Agenti, Commerciali
Anagrafica Clienti	Sedi multiple e tipizzazione	Gestione sedi legali/operative, destinazioni merce separate e distinzione cliente finale / rivenditore.	Amministrazione, Agenti
Anagrafica Clienti	Storico contratti cliente	Consultazione integrata di tutte le offerte e ordini emessi per lo specifico cliente.	Commerciali, Amministrazione
Catalogo Prodotti	Codifica univoca	Allineamento del codice prodotto tra Configuratore, Ad-Hoc ERP e Fusion PLM.	Admin, Commerciali
Catalogo Prodotti	Gestione obsoleti e visibilità	Blocco dei componenti obsoleti rilevati da PLM; flag di visibilità per migrazione progressiva da ERP.	Admin, PM / Engineering
Catalogo Prodotti	Selezione per Macrogruppi	Navigazione guidata ad albero (Macchina → Componenti → Accessori → Ricambi) con vincoli di compatibilità.	Commerciali Nuovi/Ricambi
Catalogo Prodotti	Scheda caratteristiche tecniche	Attributi tabellari strutturati: meccanici, elettrici, pneumatici, logistici e accessori correlati.	Commerciali, PM / Engineering
Catalogo Prodotti	Descrizioni standardizzate	Modello ibrido: voce principale da menu standardizzato (da Fusion) affiancata da campo note libero.	Commerciali, PM / Engineering
Offerte & Pricing	Struttura a Capitoli	Suddivisione delle offerte in capitoli gestiti come sottodistinte personalizzabili.	Commerciali
Offerte & Pricing	Pricing centralizzato e override	Prezzi sincronizzati da Ad-Hoc con possibilità di inserimento manuale per voci speciali o su misura.	Commerciali, Admin
Offerte & Pricing	Scontistica multi-livello	Calcolo sconti combinabili: per riga prodotto, per capitolo, globale (esclusa MDO), globale (inclusa MDO).	Commerciali
Offerte & Pricing	Calcoli tecnici ed economici	Calcolo automatico di potenza elettrica, consumo aria compressa, pesi, volumi e stima costi montaggio/trasporto.	Commerciali
Offerte & Pricing	Vincoli Incoterms e Paesi	Selezione assistita Incoterms (con vincoli UE/extra-UE); adattamento automatico a valuta, lingua e normative (es. UL-CSA).	Commerciali, Logistica
Offerte & Pricing	Alert informativi	Notifiche a video non bloccanti per segnalare scelte non conformi alle specifiche standard d'impianto.	Commerciali
Offerte & Pricing	Versioning e clonazione	Storico revisioni tracciato (Rev.01, Rev.02...) e funzione di duplicazione rapida di offerte esistenti.	Commerciali
Ordini & Workflow	Conversione e congelamento	Blocco del contratto commerciale alla conferma cliente e assegnazione codifica causale ordine.	Commerciali, Amministrazione
Ordini & Workflow	Notifiche multi-reparto	Alert push automatici inviati a Amministrazione, PM, Logistica e Produzione all'apertura dell'ordine.	Sistema (Cross-funzionale)
Ordini & Workflow	Esportazione verso ERP	Invio automatizzato dell'ordine confermato ad Ad-Hoc per avviare la fatturazione.	Amministrazione
Revisione Tecnica	Contratto tecnico parallelo	Gestione di una variante tecnica interna (senza valore legale) gestita dal PM con costificazione separata.	Project Management
Revisione Tecnica	Workflow decisionale modifiche	Bivio post-revisione: approvazione commerciale con nuovo contratto/RDA (impatto alto) o gestione interna (impatto basso).	PM, Commerciale
Stampe & Documenti	Layout di stampa per reparto	Generazione PDF personalizzati per Cliente, Amministrazione e Ufficio Tecnico.	Tutti i reparti
Localizzazione	Supporto multilingua	Interfaccia localizzata e traduzione automatica/manuale delle descrizioni contrattuali.	Commerciali, Agenti

Figura 4.1: Requisiti funzionali raccolti.

4.2 Requisiti non funzionali

Questa sezione, a differenza della precedente raccoglie i requisiti non funzionali. La tabella che seguirà, nella figura **??**, insieme al requisito stesso, raccoglie anche la sua categoria e la sua priorità all'interno dell'implementazione. I requisiti sono ordinati per priorità.

Categoria	Requisito Non Funzionale	Priorità	Specifica Tecnica / Obiettivo
Affidabilità	Eliminazione crash e blocchi	Critica	Stabilità operativa durante inserimento massivo righe, calcoli e stampa, azzerando i blocchi tipici di Access.
Interoperabilità	Connessione nativa ERP	Critica	Integrazione bidirezionale diretta via query SQL Server con Ad-Hoc (anagrafiche, listini ed export ordini).
Disponibilità	Architettura offline (LocalDB)	Alta	Continuità operativa garantita anche in assenza di rete o VPN tramite istanza locale SQL Server LocalDB.
Integrità Dati	Sincronizzazione automatica	Alta	Motore di sync bidirezionale tra LocalDB e SQL Server aziendale con logica di risoluzione conflitti.
Resilienza	Backup automatici in VPN	Alta	Salvataggio automatico del database locale durante le connessioni VPN per prevenire perdite o corruzioni dati.
Integrità Dati	Unificazione codici prodotto	Alta	Coerenza assoluta delle codifiche articolo tra Configuratore, Ad-Hoc ERP e Fusion PLM senza disallineamenti.
Usabilità	Workflow guidati (Wizard)	Alta	Percorsi passo-passo che guidano la compilazione per azzerare omissioni o errori di configurazione commerciale.
Usabilità	Viste orientate al ruolo (RBAC)	Alta	Maschere e sezioni della UI mostrate dinamicamente in base alle prerogative del ruolo utente loggato.
Portabilità	Framework moderno	Media	Architettura per ambiente desktop con predisposizione a target mobile.
Prestazioni	Calcolo istantaneo in-memory	Media	Elaborazione immediata a runtime di formule per potenze, consumi d'aria, pesi, volumi e stime logistiche.
Qualità Processo	Area di staging ordini	Media	Area di transito temporanea per la revisione amministrativa delle conferme d'ordine prima del caricamento contabile.
Evolutività	Predisposizione CAD / PLM	Bassa	Architettura aperta al recupero di disegni tecnici e visualizzazione di modelli 3D direttamente da Fusion.

Figura 4.2: Requisiti non funzionali raccolti.

Capitolo 5

Progettazione dell'architettura

5.1 Premessa sulle scelte architetturali

Le scelte architetturali sono dipese in primo luogo dall'infrastruttura aziendale, e successivamente dalle richieste degli utenti durante gli incontri. L'utilizzo di SQLServer, come motore di database, è stato adottato come base di tutte le scelte successive. Da questo caposaldo, la ricerca si è indirizzata verso un framework che interagisse sia con SQLServer sia con SQLite.[**?**]. La necessità di operare offline ha guidato la ricerca di un sistema installabile in locale, escludendo così le soluzioni web based. La scelta è quindi ricaduta sulla piattaforma .NET[**?**] con il linguaggio C# che, in quanto parte dell'ecosistema Microsoft, è perfettamente integrata con i database prima citati. La piattaforma .NET è espandibile mediante l'utilizzo di NuGet[**?**], che permette di aggiungere package e quindi funzionalità al progetto. I pacchetti sono sviluppati sia da Microsoft sia da terze parti, e vengono aggiornati e supportati continuamente. Questa manutenzione continua evita di incorrere in debiti tecnici futuri. L'ambiente .NET è nativamente multipiattaforma e questa proprietà soddisfa le necessità dei richiedenti. Secondo le linee guida Microsoft, per lo sviluppo delle view multipiattaforma, è consigliato l'utilizzo del package NuGet .NET MAUI[**?**]. In fase di compilazione, MAUI converte il codice sorgente in codice nativo per la piattaforma di destinazione. MAUI, integrato con il package NuGet "Community Toolkit Maui"[**?**], consente di costruire le interfacce grafiche in codice C#. Nativamente MAUI supporta solo il linguaggio Markup XAML[**?**].

5.2 Deployment

La scelta degli ambienti di sviluppo precedentemente citati porta a dover considerare il deployment dell'applicativo. Il software verrà installato in locale su ogni dispositivo, sia esso desktop o mobile, e si connetterà al database centrale. All'interno dell'azienda è presente un dominio **AD!** (**AD!**), dove ogni oggetto del dominio ha un **IP!** (**IP!**) statico. Questa configurazione permette la connessione al database, risiedente in un server all'interno del dominio, in modo agile tramite **IP!**, se si è connessi all'interno della **LAN!** (**LAN!**). Lo scambio di dati tra il frontend e il backend avviene mediante protocollo **TCP/IP!** (**TCP/IP!**). Si è valutato, per chi accede dall'esterno del dominio, di utilizzare la rete **VPN!**, stabilendo una connessione virtuale che permette di essere all'interno della **LAN!**. Il database locale, in SQLite, è strutturato come una copia del database centrale. I dati vengono allineati tra i due database mediante un processo di sincronizzazione. In assenza di connessione, verificabile tramite ping, si accede al database locale ed, una volta ristabilita la connessione, i dati vengono sincronizzati. Il diagramma dei deployment è riassunto in figura **??**.

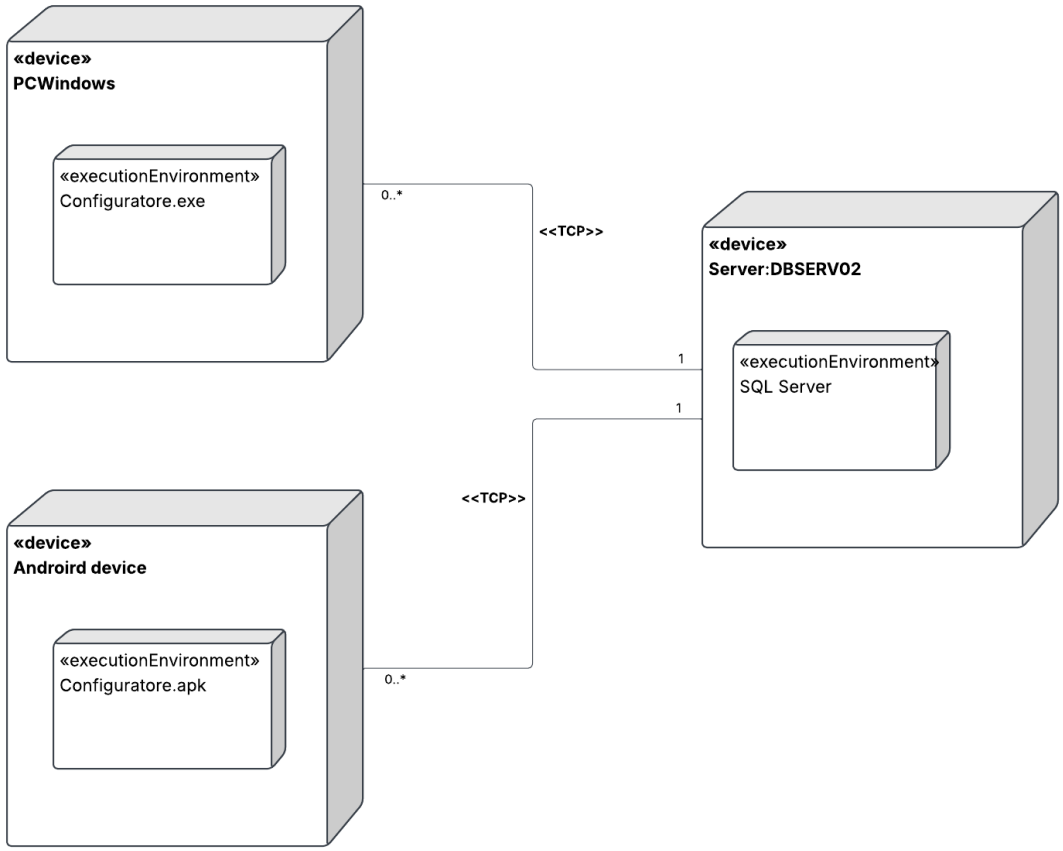


Figura 5.1: Diagramma dei deployment

Nella fase di sviluppo, per testare la struttura del database, è stato scelto di utilizzare una versione di SQLServer locale, denominata SQL Express[**?**]. SQL Express condivide con SQLServer tutte le caratteristiche principali. Questa proprietà ha permesso di costruire query e testare funzionalità localmente in modo compatibile con il database di produzione.

5.3 Package

L'intero software è stato suddiviso in tre distinti package, con responsabilità diverse. Si è cercato di seguire il principio **DRY!** (**DRY!**) e la riusabilità degli

stessi.

5.3.1 Interfaccia al database

Il package di interfaccia al database, denominato "DatabaseManagementPackage", ha il compito di gestire tutti gli aspetti legati a quest'ultimo, come:

- gestione delle connessioni ai vari database, come SQLite e SQLServer o nel caso dei test, SQL Express;
- gestione delle transazioni;
- gestione dei permessi utente, bloccando le operazioni non consentite;
- conversione, mediante **ORM!** (**ORM!**), delle tabelle del database in entity;
- mappatura di entity in file neutri, proteggendo il database;
- gestione di Repository per ogni entità, che espongono le operazioni **CRUD!** (**CRUD!**);
- gestione di una suite di test per verificare le operazioni precedenti. Questa suite è risultata fondamentale nell'evoluzione del database, validando i cambiamenti apportati;
- esporre al di fuori del package solo file neutri e la classe manager del database.

Nell'implementazione si è utilizzato il pacchetto NuGet **EF Core!** (**EF Core!**)[**?**]. Questo modulo permette una gestione efficiente delle operazioni di accesso ai dati, consentendo di astrarre le tabelle del database in classi di entity. Le entity sono state a loro volta mappate in **DTO!** (**DTO!**), proteggendo i dati. Questa seconda mappatura è stata realizzata mediante il package "Mapster"[**?**], appartenente a terze parti. La mappatura entity-**DTO!** è stata realizzata in modo bidirezionale in quanto, in alcuni casi, è necessario convertire un **DTO!** in entity per poterlo salvare nel database.

5.3.2 Libreria di componenti view

La creazione di interfacce grafiche è stata supportata da un package dedicato, che racchiude in sé factory di componenti, costanti e utilità. Il package, denominato "ViewComponentLibrary", è stato concepito a sé stante per favorire un suo impiego in implementazioni future. La libreria si appoggia a sua volta al package NuGet "Community Toolkit Maui"[**?**]. Questo package permette di scrivere le interfacce grafiche in codice C# Markup, evitando quindi l'utilizzo del linguaggio XAML[**?**], standard di sviluppo per le interfacce MAUI.

5.3.3 Modulo main e view

Il package principale del progetto è dedicato alla gestione dei componenti e dei flussi di lavoro. Il package, denominato "ConfiguratoreCommerciale", ha numerosi compiti, tra cui la gestione della Dependency Injection, la creazione delle view e dei rispettivi ViewModel. In questo package risiedono infatti i ViewModel, che si interpongono tra le view e i repository, esponendo i dati tramite proprietà osservabili (Observable Properties). La Dependency Injection, invece, permette di iniettare le dipendenze necessarie alle classi, come ad esempio i repository, le classi di logging o le view stesse se registrate. Le classi di view, utilizzando i componenti della libreria, vengono realizzate tenendo conto del dispositivo di destinazione. La differenziazione è permessa dall'utilizzo della compilazione condizionale, distinguendo il codice da compilare in base alla piattaforma di destinazione. Un esempio di tale condizione è il seguente:

```
#if ANDROID
    // codice per piattaforma Android
#elseif WINDOWS
    // codice per piattaforma Windows
#endif
```

Questa possibilità permette anche di eseguire parti di codice solo in fase di DEBUG. Gli aspetti generali gestiti dal package possono essere riassunti come segue:

- creazione delle interfacce grafiche;
- traduzione a compile time del codice C# in XAML;

- traduzione a compile time in elementi nativi per la piattaforma di destinazione;
- gestione delle interfacce mediante ViewModel, che a loro volta tramite proprietà osservabili modificano la view in risposta ad eventi;
- gestione della navigazione tra le interfacce grafiche;
- gestione dei permessi, mostrando solo i comandi che competono al ruolo dell'utente;
- gestione della Dependency Injection mediante la classe *MauiProgram.cs*;
- creazione, in fase di avvio, di una cache in **RAM!** (**RAM!**) di dati immutabili durante la sessione, come tabelle di lookup.

Il diagramma dei package è riassunto in figura **??**.

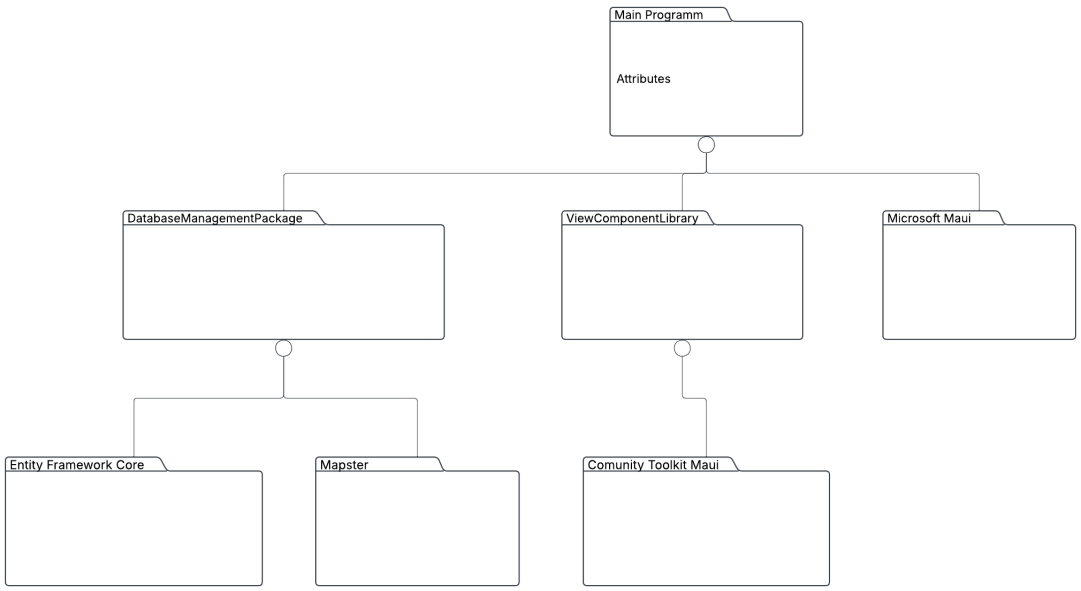


Figura 5.2: Diagramma dei package

5.4 Flussi di lavoro

I flussi di lavoro citati in figura **??** sono risultati complessi e poco efficienti. Si è quindi deciso di inserire una revisione tecnica, da parte di un utente specializzato, degli impianti venduti. Il nuovo diagramma, in figura **??**, mostra un flusso più lineare che porta a valle meno errori. Determinare i nuovi flussi lavorativi è fondamentale per progettare gli automatismi a cui sono soggette le notifiche intra-reparto.

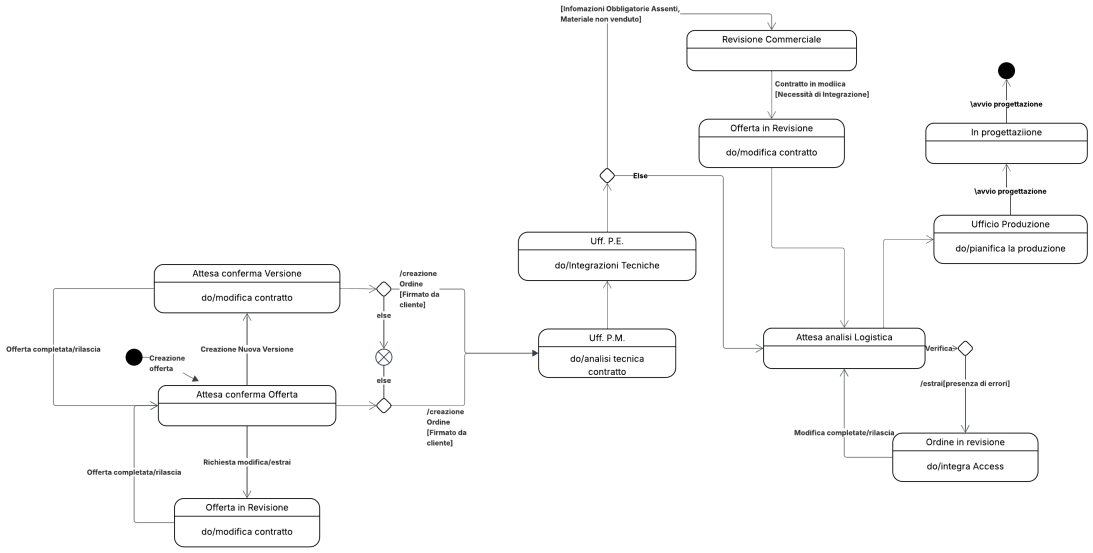


Figura 5.3: Nuovo diagramma degli stati di un contratto

Capitolo 6

Scelte progettuali

6.1 Scelta modello di sviluppo

Il modello di sviluppo scelto è basato sulla necessità di prototipare e testare con gli utenti le funzioni nell'arco dell'intero sviluppo. Si è quindi deciso di adottare un modello agile, supportato dalla creazione di test delle funzioni e dei prototipi delle interfacce. La progettazione è stata iterativa e incrementale, seguendo i canoni dell'agilità. Tutti i metodi di interazione col database sono stati testati da appositi Unit Test che ne validassero il comportamento. I test di regressione hanno permesso di convalidare le modifiche apportate al database e di conseguenza al software. Le interfacce grafiche, al contrario, non sono state supportate da test, ma prototipate e approvate da utenti scelti. La rifattorizzazione del codice è stata affrontata con regolarità seguendo il principio **DRY!**. Nel seguire tale principio, si sono create più classi di Factory e classi di utilità che unificassero quanto più possibile il codice comune.

6.2 Suddivisione e contenuto dei rilasci

Le tempistiche dei rilasci sono state decise insieme al Project Manager aziendale. Il collega è stato di supporto nella stima dei tempi di sviluppo e nella conversione del database esistente, guidando lo sviluppo attraverso le tecniche professionali. Nel primo rilascio, si è scelto di includere tutti i requisiti già presenti nel precedente

applicativo. In tale rilascio, con consegna stimata per gennaio 2027, sono presenti le seguenti funzionalità:

- gestione utenti e permessi;
- anagrafica clienti;
- creazione delle offerte;
- versioning offerte;
- creazione ordini;
- import ed export dati, da e verso altri database;
- creazione reportistica;
- gestione multilingua dell'interfaccia applicativa;
- prototipo mobile;
- modulo di configurazione prodotto.

Si è scelto di non includere la funzionalità offline, in quanto essa necessita di un database centrale stabile. La funzionalità offline verrà implementata nel secondo rilascio. Questa fase avverrà una volta che il software sarà in produzione, per poter contare su un database pressoché stabile. Nel secondo rilascio saranno quindi presenti:

- gestione revisione tecnica e differenze con contratto firmato;
- inserimento di calcoli automatici sulla base delle caratteristiche del venduto (es. potenza elettrica totale assorbita);
- gestione multi-valuta per ordini internazionali;
- apertura dei disegni PDF dal server del **PLM!**;
- report avanzati;
- implementazione database locale con logiche di sincronizzazione.

6.3 Scelta framework ORM

All'interno della sezione sull'architettura, è stata citato la scelta del framework **EF Core! ??**. Questo framework ha permesso di mappare le tabelle del database in classi mediante la tecnica **ORM!**. Questa tecnica consiste nel mappare ogni tabella del database in una classe che ne rappresenti l'entità. I legami tra le entità sono gestiti mediante la classe `Context`. L'**ORM!** ha permesso, con l'utilizzo di script PowerShell, di ricreare in modo rapido tutte le entity e il context a fronte di cambiamenti del database. Tale approccio può essere definito database-first, in quanto la progettazione avviene principalmente sul database e il software ne eredita le modifiche. Le queries sono state scritte tramite le funzioni **LINQ!** (**LINQ!**), che hanno permesso di formularle in linguaggio C#. Un esempio di query è riportato nella funzione **??**.

Listing 6.1: Esempio di query LINQ

```
1     public class Example{
2   public async Task<ClientiDTO?> GetCustomerDetailsById(int customerId,
3       CancellationToken cancellationToken = default)
4   {
5               await _permissionTable.TryGetTablePermission(typeof(ClientiDTO),
                    TypeOfOperation.READ,
6           _utils.GetCurrentUserRoleId()).ConfigureAwait(false);
7
8       Clienti? customerDetails = await _db.Clientis
9           .Where(c => c.CliId == customerId)
10          .Include(c => c.ClientiDestinazionis.Where(cd => cd.Deleted == false))
11          .ThenInclude(d => d.CdeNazioneNavigation)
12          .Include(c => c.ClientiDestinazionis.Where(cd => cd.Deleted == false))
13          .ThenInclude(d => d.CdeTipoNavigation)
14          .Include(c => c.CliBancaSedeNavigation)
15          .Include(c => c.CliRiferimentoEsternoNavigation)
16          .Include(c => c.CliRiferimentoInternoNavigation)
17          .Include(c => c.CliTipoNavigation)
18          .Include(c => c.DocumentiDocUtilizzatoreNavigations.Where(d => d.
                Deleted == false))
19          .Include(c => c.CliCorriereNavigation)
20          .Include(c => c.CliImballoNavigation)
21          .AsNoTracking()
22          .FirstOrDefaultAsync(cancellationToken)
23          .ConfigureAwait(false);
24
25      return _mapper.Map<ClientiDTO?>(customerDetails!);
26  }
27      }
```

In ottica di offline, le queries in **LINQ!** sono compatibili con il database locale SQLite se si limitano le funzioni utilizzate a quelle supportate da esso. Le funzioni compatibili sono ad esempio: *Where, Select, OrderBy, GroupBy, Join, Distinct.*

Uno degli aspetti negativi di **LINQ!** è la lentezza con grandi volumi di dati. Lo svantaggio è dovuto alla traduzione del codice in query compatibili con SQL. Si è valutato però, in caso si presentino problemi di performance, di ottimizzare il database mediante indici in prima battuta e, successivamente, ottimizzando le singole query. **EF Core!** permette di ottimizzare le queries scrivendole in SQL puro, bypassando la traduzione da **LINQ!**.

6.4 Scelta del framework grafico

Il framework grafico scelto aveva come obiettivo soddisfare il requisito della portabilità e dell'offline. Le scelte vagliate sono state:

- frontend web based, con l'utilizzo di HTML, CSS e JavaScript e interrogazione del database centrale mediante **API!** (**API!**);
- utilizzo del framework MAUI, nativamente cross-platform e integrato perfettamente con il backend.

L'opzione web based, con interfacce facilmente adattabili a qualsiasi dispositivo, ha presentato due svantaggi principali. La difficoltà principale risiedeva nell'utilizzo in assenza di connessione. In questo scenario, per funzionare, richiedeva che i file e il database fossero presenti sul dispositivo. Questa caratteristica rende difficoltoso gestire gli aggiornamenti. Il secondo motivo, non trascurabile per le tempistiche, è l'utilizzo di più linguaggi di programmazione. Quest'ultimonon massimizza l'esperienza acquisita con l'implementazione degli altri moduli. Il framework MAUI di .NET sopperisce a entrambi i problemi. Il programma è fisicamente installato sui dispositivi e il loro aggiornamento può essere gestito centralmente all'interno di uno store Microsoft. Essendo basato su .NET e avvalendosi del framework "Community Toolkit Maui", le interfacce sono scritte in C#.

6.5 Scelta del pattern di progettazione

Insieme a MAUI, Microsoft suggerisce l'utilizzo del pattern **MVVM!** (**MVVM!**).[**?**] Il modello **MVVM!** consente di separare in modo pulito le logiche di business e le relative interfacce utente. La gestione separata risolve numerosi problemi, quali la possibilità di ridisegnare le interfacce utente senza dover modificare le classi che le governano. Un secondo vantaggio è la possibilità di consentire agli sviluppatori e ai progettisti dell'interfaccia utente di collaborare più facilmente durante lo sviluppo delle rispettive parti di un'app.[**?**] Il modello **MVVM!** è composto da tre elementi fondamentali che interagiscono come in figura **??**:

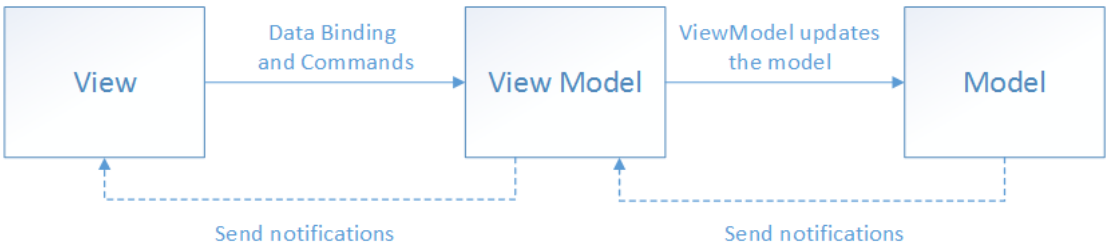


Figura 6.1: Diagramma funzionamento MVVM

Questo modello, separando la logica della view da quella di business, ha permesso di prototipare le interfacce nei primi rilasci e in futuro riprogettarle migliorando l'accessibilità e l'**UX!** (**UX!**).

Capitolo 7

Progettazione del database

7.1 Esportazione Vecchio Db

7.2 Denominazione tabelle e campi

7.3 Utilizzo di SSIS nel processo di migrazione

7.4 Ristrutturazione database

7.4.1 Eliminazione ridondanze e normalizzazione

7.4.2 Creazione tabelle di lookup

7.4.3 Aggiunta tabelle non presenti

7.4.4 Creazione tabelle di Back Up

7.4.5 Scelta di strutture Avanzate come Triggers e Stored Procedure

7.5 Schema logico finale

Capitolo 8

Creazione Model

8.1 Mapping delle classi

8.2 Struttura del package

8.2.1 Classi di Test

8.2.2 Repository

8.2.3 Classi ed Interfacce esposte

Capitolo 9

Creazione View

9.1 Utilizzo di Figma per disegno interfacce

9.2 Creazione delle Factory per le View

9.3 Scelta delle classi di ViewModel

Capitolo 10

Modulo configurazione prodotto

10.1 Scopo del modulo

10.2 Algoritmo di configurazione

10.3 Realizzazione sul database

10.4 Interfaccia grafica lato utente ed amministratore

Bibliografia

[Fig] Figma. Figma. https://www.figma.com/.

[Map] Mapster. Mapster. https://github.com/MapsterMapper/Mapster.

[Mica] Microsoft. CommunityToolkit.MAUI. https://learn.microsoft.com/it-it/dotnet/communitytoolkit/maui/.

[Micb] Microsoft. MAUI MVVM. https://learn.microsoft.com/it-it/dotnet/architecture/maui/mvvm.

[Micc] Microsoft. Microsoft Entity Framework. https://learn.microsoft.com/en-us/ef/.

[Micd] Microsoft. SQL Server. https://www.microsoft.com/sql-server.

[Mice] Microsoft. SQL Server Express. https://learn.microsoft.com/en-us/sql/database-engine/configure-windows/sql-server-express-localdb?view=sql-server-ver17.

[Micf] Microsoft. XAML language. https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/.

[.NEa] .NET. .NET. https://dotnet.microsoft.com/.

[.NEb] .NET MAUI. .NET MAUI. https://learn.microsoft.com/it-it/dotnet/maui/what-is-maui?view=net-maui-10.0.

[NuG] NuGet. NuGet package manager. https://learn.microsoft.com/it-it/nuget/.

[Rul] Rule Designer. Fusion. https://www.ruledesigner.com/it/.

[SQL] SQLite. SQLite. https://www.sqlite.org/. Lightweight database engine.

[Zuc] Zucchetti. Ad-Hoc. https://www.zucchetti.com/it-it/soluzioni/gestionali.

Acknowledgements

Optional. Max 1 page.